

LearnOpenCV - Image Classification using Transfer Learning in PyTorch

Reference: [Image Classification in PyTorch](#)

Dataset: [Caltech 256](#)

In [1]:

```
import torch, torchvision
from torchvision import datasets, models, transforms
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import time
#from torchsummary import summary

import numpy as np
import matplotlib.pyplot as plt
import os

from PIL import Image
```

C:\Users\z3534407\Anaconda3\lib\site-packages\torchvision\io\image.py:13: UserWarning: Failed to load image Python extension: Could not find module 'C:\Users\z3534407\Anaconda3\Lib\site-packages\torchvision\image.pyd' (or one of its dependencies). Try using the full path with constructor syntax.
warn(f"Failed to load image Python extension: {e}")

In [2]:

```
# Applying Transforms to the Data
image_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(size=256, scale=(0.8, 1.0)),
        transforms.RandomRotation(degrees=15),
        transforms.RandomHorizontalFlip(),
        transforms.CenterCrop(size=224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                             [0.229, 0.224, 0.225])
    ]),
    'valid': transforms.Compose([
        transforms.Resize(size=256),
        transforms.CenterCrop(size=224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                             [0.229, 0.224, 0.225])
    ]),
    'test': transforms.Compose([
        transforms.Resize(size=256),
        transforms.CenterCrop(size=224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
```

```

        [0.229, 0.224, 0.225])
    })
}

```

In [3]:

```

# Load the Data

# Set train and valid directory paths

dataset = 'caltech_10'

train_directory = os.path.join(dataset, 'train')
valid_directory = os.path.join(dataset, 'valid')
test_directory = os.path.join(dataset, 'test')

# Batch size
bs = 32

# Number of classes
num_classes = len(os.listdir(valid_directory)) #10#2#257
print(num_classes)

# Load Data from folders
data = {
    'train': datasets.ImageFolder(root=train_directory, transform=
    'valid': datasets.ImageFolder(root=valid_directory, transform=
    'test': datasets.ImageFolder(root=test_directory, transform=i
}

# Get a mapping of the indices to the class names, in order to se
idx_to_class = {v: k for k, v in data['train'].class_to_idx.items}
print(idx_to_class)

# Size of Data, to be used for calculating Average Loss and Accur
train_data_size = len(data['train'])
valid_data_size = len(data['valid'])
test_data_size = len(data['test'])

# Create iterators for the Data loaded using DataLoader module
train_data_loader = DataLoader(data['train'], batch_size=bs, shuf
valid_data_loader = DataLoader(data['valid'], batch_size=bs, shuf
test_data_loader = DataLoader(data['test'], batch_size=bs, shuffl

4
{0: 'bear', 1: 'chimp', 2: 'giraffe', 3: 'gorilla'}

```

In [4]:

```

device = torch.device("cuda:0" if torch.cuda.is_available() else
print(train_data_size, valid_data_size, test_data_size)

```

280 58 58

In [5]:

```

# Load pretrained ResNet50 Model
resnet50 = models.resnet50(pretrained=True)

```

```
resnet50 = resnet50.to(device)
```

```
In [6]: # Freeze model parameters
        for param in resnet50.parameters():
            param.requires_grad = False
```

```
In [7]: # Change the final layer of ResNet50 Model for Transfer Learning
        fc_inputs = resnet50.fc.in_features

        resnet50.fc = nn.Sequential(
            nn.Linear(fc_inputs, 256),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(256, num_classes), # Since 10 possible outputs
            nn.LogSoftmax(dim=1) # For using NLLLoss()
        )

        # Convert model to be used on GPU
        resnet50 = resnet50.to(device)
```

```
In [8]: # Define Optimizer and Loss Function
        loss_func = nn.NLLLoss()
        optimizer = optim.Adam(resnet50.parameters())
```

```
In [9]: def train_and_validate(model, loss_criterion, optimizer, epochs=25)
        ...
        Function to train and validate
        Parameters
            :param model: Model to train and validate
            :param loss_criterion: Loss Criterion to minimize
            :param optimizer: Optimizer for computing gradients
            :param epochs: Number of epochs (default=25)

        Returns
            model: Trained Model with best validation accuracy
            history: (dict object): Having training loss, accuracy and
            ...

        start = time.time()
        history = []
        best_loss = 100000.0
        best_epoch = None

        for epoch in range(epochs):
            epoch_start = time.time()
            print("Epoch: {}/{}".format(epoch+1, epochs))

            # Set to training mode
            model.train()
```

```

# Loss and Accuracy within the epoch
train_loss = 0.0
train_acc = 0.0

valid_loss = 0.0
valid_acc = 0.0

for i, (inputs, labels) in enumerate(train_data_loader):

    inputs = inputs.to(device)
    labels = labels.to(device)

    # Clean existing gradients
    optimizer.zero_grad()

    # Forward pass – compute outputs on input data using
    outputs = model(inputs)

    # Compute loss
    loss = loss_criterion(outputs, labels)

    # Backpropagate the gradients
    loss.backward()

    # Update the parameters
    optimizer.step()

    # Compute the total loss for the batch and add it to
    train_loss += loss.item() * inputs.size(0)

    # Compute the accuracy
    ret, predictions = torch.max(outputs.data, 1)
    correct_counts = predictions.eq(labels.data.view_as(p

    # Convert correct_counts to float and then compute th
    acc = torch.mean(correct_counts.type(torch.FloatTensor

    # Compute total accuracy in the whole batch and add t
    train_acc += acc.item() * inputs.size(0)

    #print("Batch number: {:03d}, Training: Loss: {:.4f},

# Validation – No gradient tracking needed
with torch.no_grad():

    # Set to evaluation mode
    model.eval()

    # Validation loop
    for j, (inputs, labels) in enumerate(valid_data_loade
        inputs = inputs.to(device)
        labels = labels.to(device)

    # Forward pass – compute outputs on input data us

```

```

        outputs = model(inputs)

        # Compute loss
        loss = loss_criterion(outputs, labels)

        # Compute the total loss for the batch and add it
        valid_loss += loss.item() * inputs.size(0)

        # Calculate validation accuracy
        ret, predictions = torch.max(outputs.data, 1)
        correct_counts = predictions.eq(labels.data.view_

        # Convert correct_counts to float and then comput
        acc = torch.mean(correct_counts.type(torch.FloatTensor

        # Compute total accuracy in the whole batch and a
        valid_acc += acc.item() * inputs.size(0)

        #print("Validation Batch number: {:03d}, Validati
    if valid_loss < best_loss:
        best_loss = valid_loss
        best_epoch = epoch

    # Find average training loss and training accuracy
    avg_train_loss = train_loss/train_data_size
    avg_train_acc = train_acc/train_data_size

    # Find average training loss and training accuracy
    avg_valid_loss = valid_loss/valid_data_size
    avg_valid_acc = valid_acc/valid_data_size

    history.append([avg_train_loss, avg_valid_loss, avg_train

    epoch_end = time.time()

    print("Epoch : {:03d}, Training: Loss - {:.4f}, Accuracy

    # Save if the model has best accuracy till now
    torch.save(model, dataset+'_model_'+str(epoch)+'.pt')

    return model, history, best_epoch

```

In [10]:

```

# Print the model to be trained
#summary(resnet50, input_size=(3, 224, 224), batch_size=bs, device=device)

# Train the model for 10 epochs
num_epochs = 10
trained_model, history, best_epoch = train_and_validate(resnet50, dataset, num_epochs)

torch.save(history, dataset+'_history.pt')

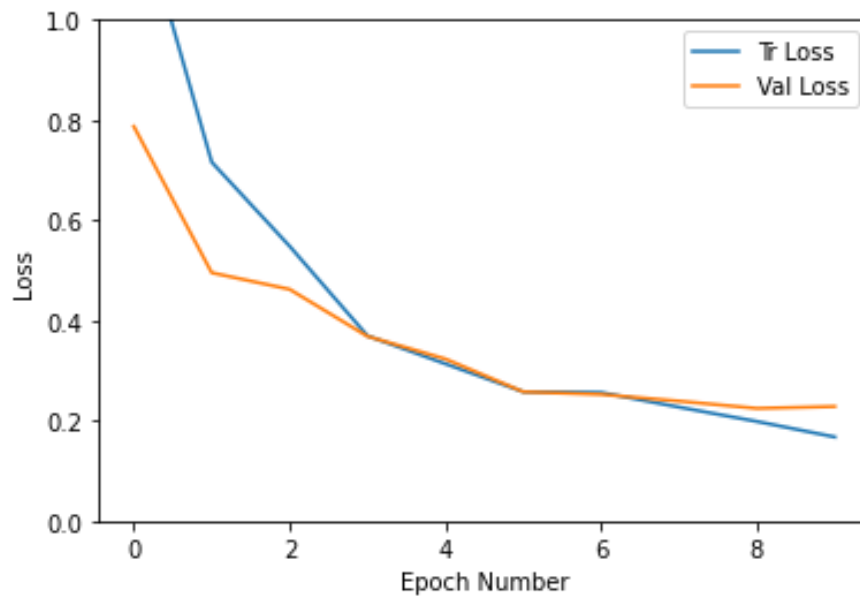
```

Epoch: 1/10

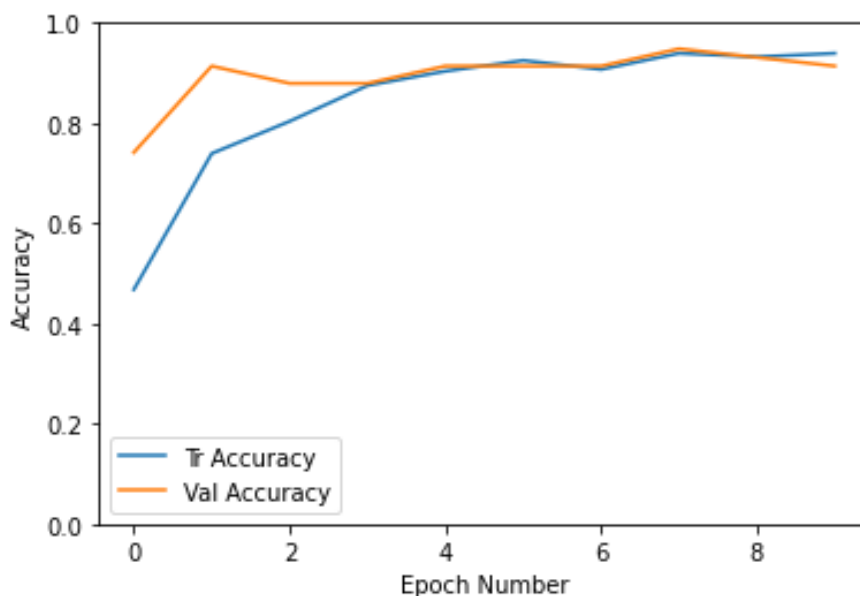
Epoch : 000, Training: Loss - 1.2597, Accuracy - 46.7857%,
 Validation : Loss - 0.7874, Accuracy - 74.1379%,

```
Time: 19.0185s
Epoch: 2/10
Epoch : 001, Training: Loss - 0.7161, Accuracy - 73.9286%,
           Validation : Loss - 0.4953, Accuracy - 91.3793%,
Time: 20.3058s
Epoch: 3/10
Epoch : 002, Training: Loss - 0.5484, Accuracy - 80.3571%,
           Validation : Loss - 0.4624, Accuracy - 87.9310%,
Time: 19.8650s
Epoch: 4/10
Epoch : 003, Training: Loss - 0.3695, Accuracy - 87.5000%,
           Validation : Loss - 0.3684, Accuracy - 87.9310%,
Time: 19.8444s
Epoch: 5/10
Epoch : 004, Training: Loss - 0.3136, Accuracy - 90.3571%,
           Validation : Loss - 0.3233, Accuracy - 91.3793%,
Time: 21.8306s
Epoch: 6/10
Epoch : 005, Training: Loss - 0.2577, Accuracy - 92.5000%,
           Validation : Loss - 0.2577, Accuracy - 91.3793%,
Time: 21.5158s
Epoch: 7/10
Epoch : 006, Training: Loss - 0.2563, Accuracy - 90.7143%,
           Validation : Loss - 0.2526, Accuracy - 91.3793%,
Time: 20.1934s
Epoch: 8/10
Epoch : 007, Training: Loss - 0.2271, Accuracy - 93.9286%,
           Validation : Loss - 0.2392, Accuracy - 94.8276%,
Time: 20.6388s
Epoch: 9/10
Epoch : 008, Training: Loss - 0.1985, Accuracy - 93.2143%,
           Validation : Loss - 0.2247, Accuracy - 93.1034%,
Time: 20.0074s
Epoch: 10/10
Epoch : 009, Training: Loss - 0.1676, Accuracy - 93.9286%,
           Validation : Loss - 0.2284, Accuracy - 91.3793%,
Time: 20.2115s
```

```
In [11]: history = np.array(history)
plt.plot(history[:,0:2])
plt.legend(['Tr Loss', 'Val Loss'])
plt.xlabel('Epoch Number')
plt.ylabel('Loss')
plt.ylim(0,1)
plt.savefig(dataset+'_loss_curve.png')
plt.show()
```



```
In [12]: plt.plot(history[:,2:4])
plt.legend(['Tr Accuracy', 'Val Accuracy'])
plt.xlabel('Epoch Number')
plt.ylabel('Accuracy')
plt.ylim(0,1)
plt.savefig(dataset+'_accuracy_curve.png')
plt.show()
```



```
In [13]: def computeTestSetAccuracy(model, loss_criterion):
    """
    Function to compute the accuracy on the test set
    Parameters
        :param model: Model to test
        :param loss_criterion: Loss Criterion to minimize
    """

    device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
    test_acc = 0.0
```

```

test_loss = 0.0

# Validation - No gradient tracking needed
with torch.no_grad():

    # Set to evaluation mode
    model.eval()

    # Validation loop
    for j, (inputs, labels) in enumerate(test_data_loader):
        inputs = inputs.to(device)
        labels = labels.to(device)

        # Forward pass - compute outputs on input data using
        outputs = model(inputs)

        # Compute loss
        loss = loss_criterion(outputs, labels)

        # Compute the total loss for the batch and add it to
        test_loss += loss.item() * inputs.size(0)

        # Calculate validation accuracy
        ret, predictions = torch.max(outputs.data, 1)
        correct_counts = predictions.eq(labels.data.view_as(p

        # Convert correct_counts to float and then compute th
        acc = torch.mean(correct_counts.type(torch.FloatTensor

        # Compute total accuracy in the whole batch and add t
        test_acc += acc.item() * inputs.size(0)

    print("Test Batch number: {:03d}, Test: Loss: {:.4f},

    # Find average test loss and test accuracy
    avg_test_loss = test_loss/test_data_size
    avg_test_acc = test_acc/test_data_size
    print("Test accuracy : " + str(avg_test_acc))

```

In [14]:

```

def predict(model, test_image_name):
    """
    Function to predict the class of a single test image
    Parameters
        :param model: Model to test
        :param test_image_name: Test image
    """

    transform = image_transforms['test']

    test_image = Image.open(test_image_name)
    plt.imshow(test_image)

    test_image_tensor = transform(test_image)

```



```

if torch.cuda.is_available():
    test_image_tensor = test_image_tensor.view(1, 3, 224, 224)
else:
    test_image_tensor = test_image_tensor.view(1, 3, 224, 224)

with torch.no_grad():
    model.eval()
    # Model outputs log probabilities
    out = model(test_image_tensor)
    ps = torch.exp(out)

    topk, topclass = ps.topk(3, dim=1)
    cls = idx_to_class[topclass.cpu().numpy()[0][0]]
    score = topk.cpu().numpy()[0][0]

    for i in range(3):
        print("Prediction", i+1, ":", idx_to_class[topclass.c

```

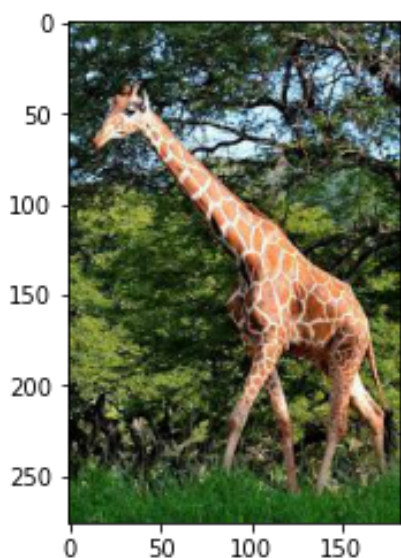
In [15]:

```

# Test a particular model on a test image
dataset = 'caltech_10'
model = torch.load("{}_model_{}.pt".format(dataset, best_epoch))
predict(model, './girrafe.jpg')

```

Prediction 1 : giraffe , Score: 0.98704207
 Prediction 2 : chimp , Score: 0.0054601086
 Prediction 3 : gorilla , Score: 0.004230091



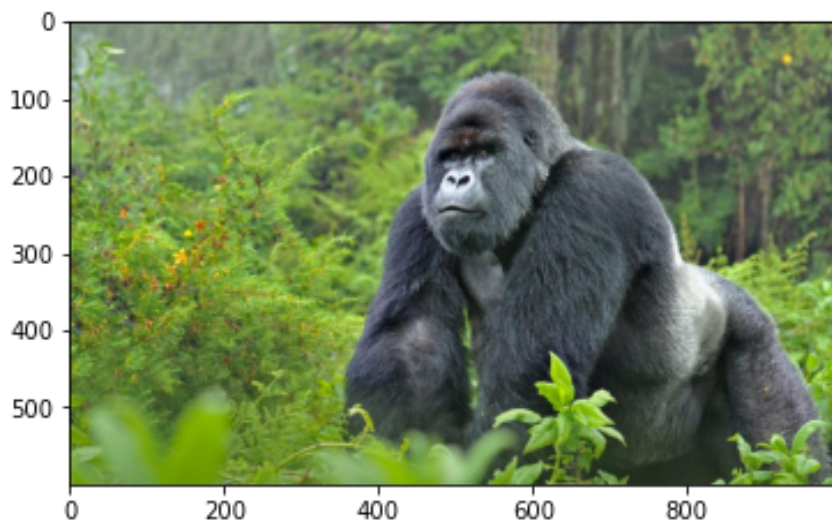
In [16]:

```

# Test a particular model on a test image
dataset = 'caltech_10'
model = torch.load("{}_model_{}.pt".format(dataset, best_epoch))
predict(model, './gorilla.jpg')

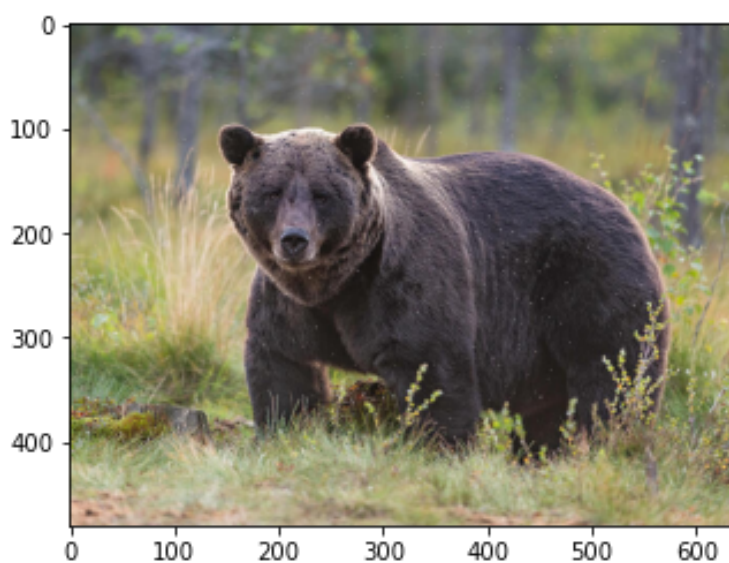
```

Prediction 1 : gorilla , Score: 0.9807567
 Prediction 2 : chimp , Score: 0.013941506
 Prediction 3 : bear , Score: 0.0052058976



```
In [17]: # Test a particular model on a test image
dataset = 'caltech_10'
model = torch.load("{}_model_{}.pt".format(dataset, best_epoch))
predict(model, './bear.jpg')
```

```
Prediction 1 : bear , Score: 0.9922402
Prediction 2 : gorilla , Score: 0.0062123924
Prediction 3 : chimp , Score: 0.001357519
```



```
In [18]: computeTestSetAccuracy(model, loss_func)
```

```
Test Batch number: 000, Test: Loss: 0.2896, Accuracy: 0.9062
Test accuracy : 0.5
Test Batch number: 001, Test: Loss: 0.1447, Accuracy: 0.9615
Test accuracy : 0.9310344704266252
```